

NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering

B.Tech Project I – Mid-Semester Evaluation



AI-Based Reliable Droplet Routing for MEDA Biochips Using Deep Reinforcement Learning

Submitted by

Kashika Puri (2023UCA1885)
Gokul Kumar (2023UCA1919)
Mohammed Eshaan (2023UCA1960)

B.Tech, Class of 2027

Under the Guidance of

Dr. Ankur Gupta
*Department of Computer Science
and Engineering*

NSUT

September 2026

Signature of Supervisor

Contents

1	Introduction	1
1.1	Background: From Digital Microfluidics to MEDA	1
1.2	Droplet Routing and the Reliability Problem	1
1.3	Scope of This Project and Work Completed	2
2	Motivation	2
2.1	Microelectrode Degradation Is Use-Dependent	2
2.2	Why Classical Shortest-Path Routing Is Not Enough	3
2.3	Why Deep Reinforcement Learning	3
3	Literature Survey	3
3.1	Biochip Architectures	3
3.2	Classical Droplet Routing	4
3.3	Reliability and Error Recovery on MEDA	4
3.4	Reinforcement-Learning-Based Droplet Routing	4
3.5	Sample Preparation on MEDA	5
3.6	Research Gap	5
4	Problem Statement	6
5	Objective and Methodology	6
5.1	Objectives	6
5.2	Overall Framework	7
5.3	Phase 1 – MEDA Simulator	7
5.4	Phase 2 – Classical Baselines	8
5.5	Phase 3 – Deep Reinforcement Learning Router	8
5.6	Phase 4 – Curriculum Learning	9
5.7	Evaluation Protocol	9
6	Simulation Platform and Requirements	9
6.1	Software Stack	9
6.2	Hardware	9
7	Preliminary Results and Discussion	10
7.1	Experiment 1 – Behaviour of the Two Baselines	10
7.2	Experiment 2 – Success Versus Degradation Level	10
7.3	Experiment 3 – Chip Lifetime	11
7.4	Experiment 4 – First PPO Agents	12
8	Conclusion and Future Work	13
8.1	Conclusion	13
8.2	Future Work	13

1 Introduction

1.1 Background: From Digital Microfluidics to MEDA

Microfluidic biochips, also called *labs-on-a-chip*, are small devices that can carry out lab procedures like diluting a sample, mixing reagents and detecting a result, all on one chip and with very little human effort. They are used in point-of-care testing, DNA and protein analysis and drug screening, and they got a lot of attention during the COVID-19 pandemic [3]. There are broadly three types [3]. *Continuous-flow* biochips move liquid through fixed channels using pumps and valves, so each chip is basically built for one protocol. *Digital microfluidic biochips* (DMFBs) work differently: they move small separate droplets (in the nanolitre range) over a 2D grid of electrodes using electrowetting-on-dielectric (EWOD) [6, 7]. When the electrode next to a droplet is switched on, the droplet gets pulled onto it. DMFBs can be reprogrammed, but their electrodes are big and fixed in size, so droplet volumes are coarse and there is not much sensing on the chip.

The *Micro-Electrode-Dot-Array* (MEDA) architecture [8–10] was introduced to fix these problems. Instead of a few large electrodes, a MEDA chip has a large number of tiny identical microelectrodes, and each one has its own CMOS circuit for switching and sensing. A droplet is just a group of neighbouring microelectrodes that are turned on together (Figure 1.1). Because of this, droplets can have different sizes and shapes, they can move diagonally as well as up/down/left/right, and they can even be reshaped while moving. The built-in sensors also tell us where a droplet actually is, which means the chip can be controlled in a closed loop [11].

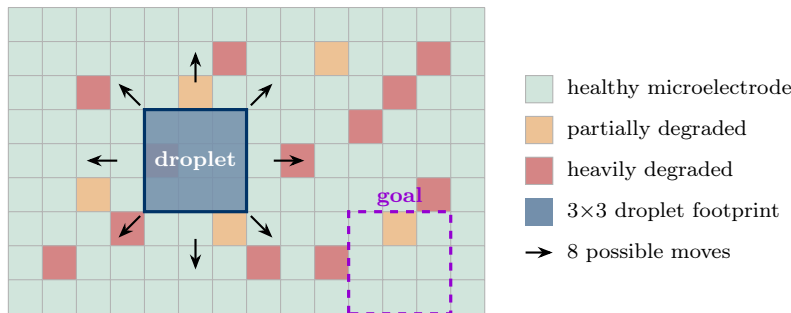


Figure 1.1: Top view of a MEDA biochip as modelled in this project. A droplet occupies a 3×3 block of microelectrodes and can move one microelectrode in any of eight directions. Microelectrodes wear out unevenly with use; a droplet sitting on weak microelectrodes may fail to move.

1.2 Droplet Routing and the Reliability Problem

To run a bioassay on a biochip, it has to be broken down into a series of droplet movements. For example, a sample droplet goes from a reservoir to a mixer, then the mixed droplet goes to a detector, and so on. Deciding the path for each of these movements is called *droplet routing*. On MEDA this is more flexible than on a DMFB but also harder, since a droplet can move in eight directions and can change its shape [17, 18].

The other big issue is reliability. Every time an EWOD electrode is switched on, a little bit of charge gets trapped in its dielectric layer [12]. Over time this trapped charge weakens the force that moves the droplet, so an electrode that is used a lot slowly loses its ability to

move droplets. If a droplet lands on a worn-out region it can get stuck, the bioassay misses its deadline, and after enough wear the chip has to be thrown away. Normal routers like A* treat every electrode as equally good and keep sending droplets through the same shortest paths, which is exactly what wears those paths out faster.

1.3 Scope of This Project and Work Completed

In this project, which runs over Semester 7 and Semester 8, we are building an *AI-based droplet router* for MEDA biochips. It uses deep reinforcement learning (DRL) to find paths that are short and also reliable on a chip whose microelectrodes wear out. Our base paper is Elfar *et al.* (IEEE TCAD 2023) [1], who showed that a DRL agent that can see the microelectrode health map learns to route around damaged areas. For the chip model we follow Liang *et al.* (ICML 2021) [2]. We have split the project into five stages, starting with a single-droplet router this semester and ending with multi-droplet routing driven by real sample-preparation workloads next semester (Section 8.2).

Till the mid-semester evaluation, we have done the following:

1. **MEDA simulator.** We wrote our own MEDA chip simulator in Python (using Gymnasium) where electrodes wear out randomly based on how much they are used. We checked it with 20 unit tests.
2. **Two classical baselines.** Normal A*, and a *health-aware* A* router that we designed ourselves, where the cost of each step is the expected number of actuation cycles.
3. **Baseline experiments.** We measured success rate at different degradation levels (2,500 chips in total) and ran a chip-lifetime test (10 chips, 3,000 tasks each).
4. **First DRL agents.** We trained two PPO agents with a CNN policy, one with the health map and one without, and compared them with both baselines on 500 chips they had never seen.

2 Motivation

2.1 Microelectrode Degradation Is Use-Dependent

Because of charge trapping, how healthy a microelectrode is depends on how many times it has been switched on [1, 12]. Following [1, 2], we model the health of microelectrode e after n_e actuations as

$$h_e(n_e) = d_e^{\lfloor n_e/N_{\text{th}} \rfloor}, \quad d_e \in [0.6, 1.0], \quad (2.1)$$

where d_e is a decay factor that is different for each electrode (to represent manufacturing variation) and $N_{\text{th}} = 50$ is the number of actuations after which the damage starts to show. When a droplet tries to move, the move works with a probability equal to the average health of the microelectrodes under it. So basically, *the paths we pick now decide what the chip will look like later*. If a router keeps using the same short corridor, that corridor wears out quickly. If it spreads the droplet traffic around, the chip stays usable for longer. Every extra actuation takes something away from the chip's lifetime, which is also why reducing actuations matters in programmable microfluidic devices [5].

2.2 Why Classical Shortest-Path Routing Is Not Enough

Most existing routing methods for DMFBs and MEDA, such as network-flow and negotiation-based routers, ILP models and A*-style search [13, 14, 17, 18, 20], try to minimise path length, completion time or contamination on a chip that is assumed to never change. When electrodes start degrading, they have three problems:

- They assume every move works, so a path that looks short on paper can actually be slow if it goes over weak electrodes.
- They only look at the present. Even if they use the current health values, they have no way of predicting the wear that their own paths will cause later.
- Exact methods that use solvers [17] become very slow for bigger chips, and have to be solved again every time the chip changes.

Our own experiments (Chapter 7) show this clearly: when 70% of the microelectrodes can degrade, plain A* reaches the goal in time in only 22.8% of the tasks.

2.3 Why Deep Reinforcement Learning

Routing on a chip that keeps degrading is a problem where we have to make a sequence of decisions under uncertainty: every move may or may not succeed, and every move also changes the chip a little. This is the kind of problem reinforcement learning (RL) is meant for. An RL agent learns what to do by trial and error inside a simulator. With a convolutional network it can take the whole health map as input, and once it is trained it only needs one forward pass to pick each move, which fits well with MEDA’s fast (millisecond-level) actuation cycle [3]. Earlier work has already shown that DRL routing works on DMFBs [22, 24] and on MEDA [1, 2, 23]. But a few practical questions are still open. For example, how good do classical routers get if they are also given the health information? Does curriculum learning make training faster on MEDA? And how do routing policies interact with the sample-preparation steps that actually create the routing tasks? These are the questions that led us to this project.

3 Literature Survey

3.1 Biochip Architectures

A good starting point for our survey was the recent review by Das *et al.* [3], which covers 188 papers on continuous-flow, digital and MEDA biochips. Table 3.1 lists the differences between these architectures that are important for routing. MEDA microelectrodes are about 9–16 times smaller than a DMFB electrode, so one DMFB cell is roughly equal to a 3×3 or 4×4 block of MEDA microelectrodes. That is why we use a 3×3 droplet in our simulator.

Table 3.1: Comparison of biochip architectures (adapted from [3]).

Aspect	Continuous-flow	DMFB	MEDA
Fluid handling	Channels, pumps, valves	Discrete droplets on electrodes	Droplets on a microelectrode array
Droplet size/shape	–	Fixed by electrode size	Variable, re-shapeable
Droplet movement	Along channels	4 directions	8 directions + reshaping
Sensing	External	Limited, external	Integrated per microelectrode
Spacing rule	Channel crossings	Spacing ≥ 2 cells	Spacing $\geq W+1, H+1$

3.2 Classical Droplet Routing

The early DMFB routers treated each droplet movement as a *net* and made sure that droplets from different nets never come too close and accidentally merge (static and dynamic fluidic constraints) [13]. BioRoute [14] used network flow for global routing followed by negotiation-based detailed routing, and contamination-aware routing [15] picked among the k shortest paths to reduce leftover residue. For MEDA, Chen *et al.* [16] proposed a 3D-A* router that handles droplets of different sizes and also uses diagonal moves. Keszocze *et al.* [17] solved MEDA routing *exactly* using a formal solver, which gives optimal answers but takes a lot of time. Howladar *et al.* worked on homogeneous routing with detours and compaction [18] and on avoiding cross-contamination [19], and Das and Chakraborty [20] used droplet reshaping along with a Ford–Fulkerson capacity graph to route the nets that failed in the first pass. One thing common to all of these is that they assume every electrode works perfectly.

3.3 Reliability and Error Recovery on MEDA

Since MEDA microelectrodes can sense droplets, errors can be caught and fixed while the chip is running. Zhong *et al.* [11] proposed an adaptive error-recovery method based on droplet-aliquot operations and predictive analysis. Elfar *et al.* [21] used formal methods to build adaptive routing strategies that take degraded microelectrodes into account. Their guarantees are strong, but the models get very large as the chip grows. From these papers it is clear that routing needs to be *health-aware*, and that we need methods that scale better, which is where learning comes in.

3.4 Reinforcement-Learning-Based Droplet Routing

Table 3.2 lists the RL-based routers closest to our work. Liang *et al.* [22] were the first to use DRL for droplet routing on DMFBs with degrading electrodes, and they showed that the agent can adapt in cases where fixed routers fail. Elfar *et al.* then applied this idea to MEDA [23], and in our base paper [1] they added the *microelectrode health map* to what the agent sees and trained it with a CNN-based policy. This gave shorter and more reliable routes than the baseline routers on degraded chips. Liang *et al.* [2] extended MEDA routing to multiple droplets using multi-agent RL and tested it on a real fabricated chip. Their open-source `meda-env` simulator is what we based our chip model on. Kawakami *et al.* [24] looked at DRL routing on DMFBs with faulty electrodes, and Dutta *et al.* [25] used a dueling network on hexagonal DMFBs. Most recently, Liu *et al.* [4] showed that

curriculum learning, where the agent is trained on easy cases first, makes DRL routing on normal DMFBs faster to train.

Table 3.2: Reinforcement-learning approaches to droplet routing.

Work	Platform	Method	Relevance / gap
Liang et al. [22]	DMFB	DRL, degrading electrodes	Fixed electrodes; 4-direction moves
Elfar et al. [1, 23]	MEDA	PPO + CNN, health map	Base paper ; baseline is health-unaware
Liang et al. [2]	MEDA	Multi-agent RL	Chip model we follow; code needs TensorFlow 1
Kawakami et al. [24]	DMFB	DRL, erroneous electrodes	Not MEDA
Dutta et al. [25]	Hex. DMFB	Dueling DQN	Not MEDA
Liu et al. [4]	DMFB	DRL + curriculum	Not yet applied to MEDA

3.5 Sample Preparation on MEDA

In a real chip, routing tasks are not random. They come from the bioassay itself, and a big part of most bioassays is *sample preparation*, which means diluting a sample down to a target concentration. MEDA can mix in ratios other than 1:1, and several algorithms make use of this. WSPM [26] builds weighted dilution graphs to use less sample and create less waste, MRCM [27] uses ILP when there are multiple reactants, and FacDA, TVODA, STWM and MTWM improve factor-based mixing [3]. For normal DMFBs, REMIA [28] and WARA [29] reduce reactant use and waste. All of these focus on saving reagents, but none of them looks at how the droplet movements they create end up wearing out the chip.

3.6 Research Gap

After going through the literature, we found four gaps that our project tries to fill:

1. DRL routers on MEDA are usually compared only with baselines that ignore electrode health. Nobody has tried a classical router that also uses the health map, so we can't tell how much of the DRL improvement is due to learning and how much is just due to having extra information.
2. Curriculum learning has been tried for DMFB routing [4], but not for MEDA routing.
3. DRL routing that makes use of MEDA's ability to *reshape* droplets has not been explored yet.
4. No one has measured how the choice of sample-preparation algorithm affects chip lifetime through the routing work it creates.

4 Problem Statement

Build a droplet router for MEDA biochips that *reliably* gets droplets to their destination before the bioassay deadline, on a chip whose microelectrodes wear out with use, and that keeps the chip working for as many routing tasks as possible.

Formal statement. We take a MEDA chip as an $H \times W$ grid of microelectrodes, where each electrode has a health value $h_e \in [0, 1]$ that changes according to Eq. (2.1). A droplet centred at $x = (y, x)$ covers a 3×3 footprint $F(x)$. In each actuation cycle t , the router picks a direction $a_t \in \mathcal{A} = \{N, NE, E, SE, S, SW, W, NW\}$. The move to $x_t + \delta(a_t)$ works with probability

$$p(x_t) = \frac{1}{|F(x_t)|} \sum_{e \in F(x_t)} h_e, \quad (4.1)$$

and if it fails the droplet just stays at x_t . Either way, the microelectrodes under the droplet get actuated, so their counters n_e go up. A *routing task* is a start s and a goal g , with a deadline of

$$T(s, g) = \lceil 1.5 d_\infty(s, g) \rceil, \quad d_\infty(s, g) = \max(|s_y - g_y|, |s_x - g_x|), \quad (4.2)$$

where d_∞ is the length of the shortest path when eight directions are allowed. A task is *successful* if the droplet covers the goal within T cycles. We use the factor 1.5 to represent the time budget of one bioassay step [3]. If a chip has to run a sequence of tasks $(s_k, g_k)_{k=1}^K$, we want a routing policy π that

$$\max_{\pi} \frac{1}{K} \sum_{k=1}^K \Pr [\text{task } k \text{ succeeds} \mid \pi], \quad (4.3)$$

Note that the chip seen by task k depends on all the routes taken before it. Since both the moves and the wear are random, we model this as a Markov decision process (MDP) and solve it using DRL (Section 5.5).

Research questions.

- RQ1.** How much does knowing microelectrode health improve routing, for classical and learned routers, as the chip degrades?
- RQ2.** Can a DRL router trained in simulation match or beat a strong health-aware A* baseline on unseen chips?
- RQ3.** Does curriculum learning over chip size and degradation make training faster or the policy better on MEDA?
- RQ4.** Which sample-preparation strategy, through the routing workload it creates, keeps a MEDA chip usable the longest? (Semester 8)

5 Objective and Methodology

5.1 Objectives

1. Build a MEDA biochip simulator with use-dependent microelectrode degradation, compatible with standard RL libraries.
2. Implement standard A* and a health-aware A* router as baselines.

3. Train a PPO-based DRL router with the health map as input and evaluate it against both baselines on unseen chips.
4. Study curriculum learning for MEDA routing.
5. Extend the router to multiple droplets and droplet reshaping, and drive it with sample-preparation workloads to measure chip lifetime (Semester 8).

5.2 Overall Framework

Figure 5.1 shows how the whole system fits together. The simulator creates chips and tasks. A router (either classical or learned) picks a direction in every actuation cycle, and the simulator then applies the random move and the wear that comes with it. Finally, all methods are tested on the same set of chips that none of them has seen before.

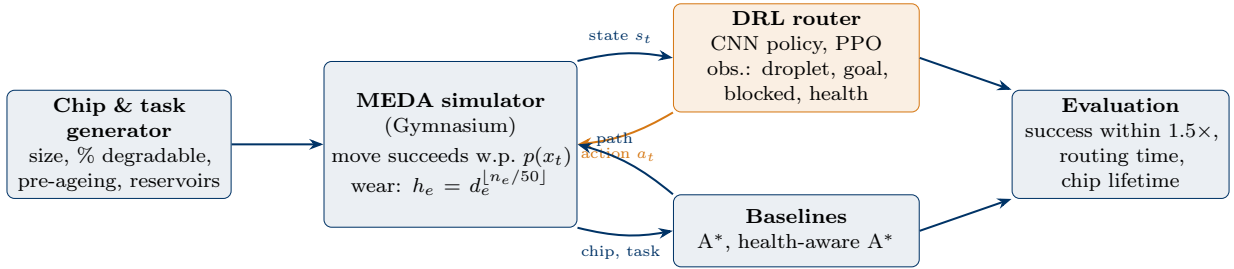


Figure 5.1: Overall framework: simulator, classical and learned routers, and a common evaluation protocol.

5.3 Phase 1 – MEDA Simulator

The original `meda-env` code from Liang *et al.* [2] needs TensorFlow 1 and an old version of Stable-Baselines, and we could not get it to run. So we rewrote the chip model ourselves as a Gymnasium environment [34]. The main parts are:

- **Chip.** An $H \times W$ array of health values. A fraction f of the microelectrodes can degrade, each with its own $d_e \sim \mathcal{U}[0.6, 1.0]$; the others have $d_e = 1$ and never wear out. A chip can start fresh or *pre-aged* ($h_e = d_e^{k_e}$ with $k_e \sim \mathcal{U}\{0, \dots, 15\}$), which is like a chip that has already been used for a while.
- **Droplet and moves.** The 3×3 droplet moves one microelectrode per cycle in any of the eight directions. A move that would take it off the chip does nothing. A move works with probability $p(x_t)$ from Eq. (4.1), and if it fails, the droplet tries again in the next cycle.
- **Wear.** After every cycle, the nine microelectrodes under the droplet count as actuated, and after every $N_{th} = 50$ actuations an electrode’s health gets multiplied by d_e (Eq. (2.1)).
- **Tasks.** Either random start and goal positions that are at least six microelectrodes apart, or, for the lifetime experiment, tasks between six fixed spots that act as reservoirs and mixers.

We made three changes compared to `meda-env` to make the model closer to real MEDA hardware: the droplet moves by one microelectrode per step instead of a whole cell, the droplet has to exactly overlap the goal to count as success, and health is updated after every step, so wear also builds up during a single task.

5.4 Phase 2 – Classical Baselines

A*. This is the standard A* search [30] over droplet positions, where every move costs 1 and the Chebyshev distance d_∞ is used as the heuristic. It does not look at electrode health at all.

Health-aware A* (our design). Since a failed move is just tried again, the number of cycles needed to get out of position x follows a geometric distribution with mean $1/p(x)$. So we set the cost of leaving x as

$$c(x) = \frac{1}{\max(p(x), 10^{-3})}, \quad \mathbb{E}[\text{cycles along path } x_0, \dots, x_L] = \sum_{i=0}^{L-1} \frac{1}{p(x_i)}, \quad (5.1)$$

This way the planner minimises the *expected routing time* on the chip as it is right now. Because $p \leq 1$, each step costs at least 1, so d_∞ is still an admissible heuristic and the search still finds the optimal path. We compute the move probability for all positions at once in $O(HW)$ time using a 2D prefix sum. If every electrode is healthy, this method gives exactly the same result as normal A*. Both baselines plan the full path once at the start of a task and then follow it, retrying any move that fails.

5.5 Phase 3 – Deep Reinforcement Learning Router

MDP. We follow [1] here. The *state* is a $4 \times H \times W$ tensor with four channels: the droplet, the goal, blocked cells (kept empty for now, it will hold other droplets in Semester 8) and the microelectrode health map. The *action* is one of the eight directions. The *reward* is

$$r_t = \begin{cases} +1 & \text{if the droplet reaches the goal,} \\ -0.05 & \text{if the Euclidean distance to the goal decreased,} \\ -0.1 & \text{otherwise (including failed moves),} \end{cases} \quad (5.2)$$

This encourages the agent to reach the goal quickly and punishes it for getting stuck. For comparison we also train an agent without the health channel (only 3 channels), which is the setup used in [2].

Learning algorithm. We use Proximal Policy Optimization (PPO) [31] from the Stable-Baselines3 library [33]. PPO maximises the clipped objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad \rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \quad (5.3)$$

where the advantages $\hat{A}_t$ come from generalised advantage estimation, and a value loss and an entropy bonus are added to it. The clipping stops the policy from changing too much in one update, which is why PPO is stable and fairly easy to tune.

Network. The default CNN in Stable-Baselines3 is meant for images of at least 36×36 pixels, which is bigger than our chips. So we wrote a small CNN that works on chip-sized grids: three 3×3 convolution layers (32, 64 and 64 channels, padding 1, ReLU), then a 256-unit fully connected layer, and then separate 128-unit heads for the policy and the value.

Training. In every episode the agent gets a new random pre-aged chip and a new task, so it has to learn a general strategy instead of memorising one chip. While training, an episode

is cut off after $3 \times d_\infty(s, g)$ steps, but during testing we use the stricter $1.5\times$ deadline from Eq. (4.2). All the hyper-parameters are in Table 6.1.

5.6 Phase 4 – Curriculum Learning

If we train straight away on big, badly degraded chips, learning is slow because the agent hardly ever reaches the goal in the beginning, so it gets almost no positive reward. Following Liu *et al.* [4] and the general idea of curriculum learning [32], we first train on small chips with little degradation and then slowly increase the chip size and the fraction of degradable microelectrodes. As far as we know, this has not been done for MEDA routing before. To compare fairly, the curriculum and non-curriculum agents need to get the same total number of training steps and be tested on the same unseen chips.

5.7 Evaluation Protocol

We test every method on exactly the *same* unseen chips and tasks, generated with fixed random seeds that are different from the ones used in training. We look at: (i) **success rate**, i.e. the fraction of tasks finished within the $1.5\times$ deadline; (ii) **routing time**, the average number of actuation cycles (including retries) for successful tasks; and (iii) for the lifetime test, the **number of tasks before the first failure** and the **overall success rate** over a long sequence of tasks on the same chip.

6 Simulation Platform and Requirements

6.1 Software Stack

- **Python 3** with **NumPy** for the chip model and the planners, and **Matplotlib** for the plots.
- **Gymnasium** (≥ 0.29) [34]: the standard interface for RL environments. Our environment passes Gymnasium’s built-in environment checker.
- **PyTorch** (≥ 2.0) [35] and **Stable-Baselines3** (≥ 2.3) [33]: for PPO and our custom CNN.
- **pytest**: 20 unit tests that check chip creation, move probabilities, wear, rewards, episode cut-off, observations and the planners (for example, that health-aware A* goes around a weak band and gives the same path as A* on a healthy chip).

6.2 Hardware

- **Classical experiments** (Experiments 1–3) only need a CPU and take anywhere from a few seconds to about two minutes.
- **PPO training** (Experiment 4) was done on Google Colab with an NVIDIA T4 GPU (16 GB). Training one agent for 300k steps with 8 parallel environments took about 11–12 minutes.
- For the longer training runs (1–2 M steps) and the multi-agent experiments planned later, we will also use Colab GPU sessions.

Code organisation and reproducibility. Our code is a small Python package with a simulator module (`env.py`), a baselines module (`routers.py`), four experiment scripts

that take command-line options (chip size, degradation fraction, number of chips, deadline slack), and a Colab notebook that runs everything from start to end. All chips and tasks come from fixed seeds, and in each experiment every router gets its own identical copy of the same chip, so the comparisons are one-to-one.

Table 6.1: Simulation and training parameters used in the preliminary experiments.

Parameter	Value
Chip size	30×30 (Exp. 1–3); 12×12 (Exp. 4)
Droplet	3×3 microelectrodes, 8 directions, 1 microelectrode per step
Degradation	$d_e \sim \mathcal{U}[0.6, 1.0]$ on a fraction f of electrodes; $N_{th} = 50$
Pre-ageing	$k_e \sim \mathcal{U}\{0, \dots, 15\}$ degradation steps (Exp. 1, 2, 4)
Deadline	$1.5 \times$ shortest path (test); $3 \times$ (training episodes)
PPO	8 envs, $n_{steps} = 256$, batch 512, 4 epochs, learning rate 3×10^{-4} , $\gamma = 0.99$, entropy coef. 0.01
Policy network	3 conv layers (3×3 ; $32/64/64$) $\rightarrow$ 256 FC; policy and value heads 128
Training budget	300k environment steps per agent, one seed

7 Preliminary Results and Discussion

7.1 Experiment 1 – Behaviour of the Two Baselines

Figure 7.1 shows one task on a pre-aged 30×30 chip where 50% of the microelectrodes can degrade. Both routers take 23 moves, but A* goes straight through several weak microelectrodes while health-aware A* curves around them. Because of this, the expected routing time drops from about 33 to about 29 cycles (around 12% faster), even though the path is not any longer.

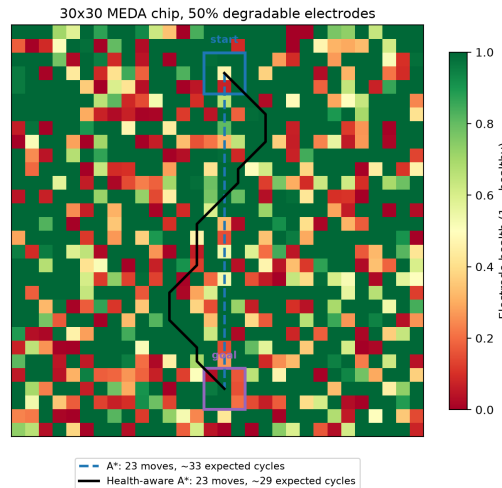


Figure 7.1: Paths of A* (dashed) and health-aware A* (solid) on a partly worn chip. Colour shows microelectrode health.

7.2 Experiment 2 – Success Versus Degradation Level

For each degradation level, we made 500 pre-aged 30×30 chips with one random task each, and ran both routers on identical copies of each chip. The results are in Table 7.1.

Table 7.1: Success rate within the $1.5\times$ deadline and mean routing time of successful tasks (500 chips per level).

Degradable electrodes	Success rate		Routing time (cycles)	
	A*	Health-aware A*	A*	Health-aware A*
0%	1.000	1.000	14.6	14.6
10%	0.994	1.000	15.4	14.8
30%	0.918	0.970	17.5	16.3
50%	0.634	0.848	19.2	18.6
70%	0.228	0.492	18.2	19.8

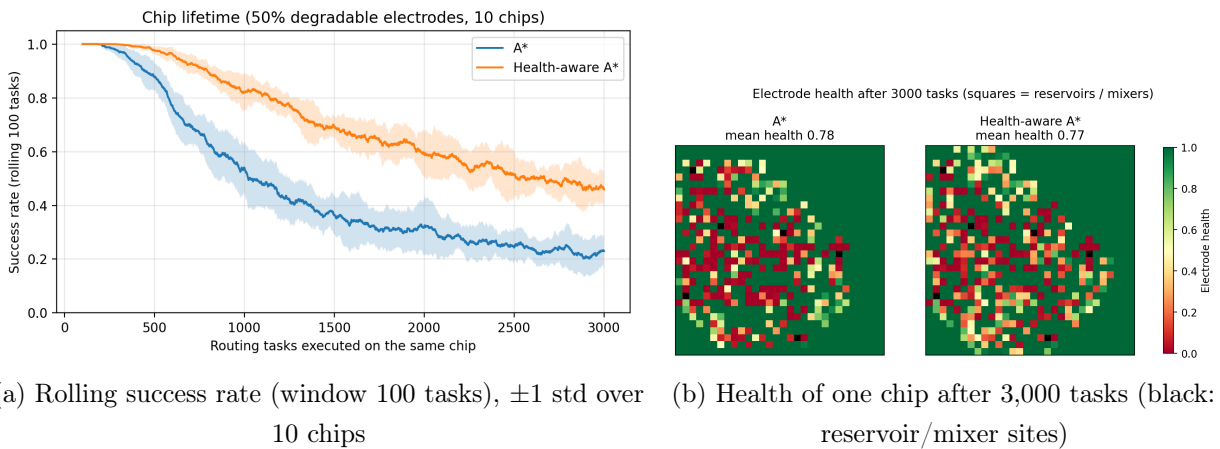
Observations. The planned paths of both routers have almost the same length (14.2–14.6 moves at every level), so the difference in results comes entirely from *where* the path goes. Plain A* gets much worse as degradation increases, dropping from 91.8% at 30% to only 22.8% at 70%. The benefit of using health information grows as the chip gets more worn: +5.2 points at 30%, +21.4 at 50% and +26.4 at 70%, which means more than double the success rate at 70%. Health-aware A* has a slightly higher routing time at 70%, but that is because it also manages to finish harder tasks that A* fails on, and those take longer.

7.3 Experiment 3 – Chip Lifetime

In an actual bioassay, droplets keep moving between the same reservoirs and mixers again and again. To test this, we took 10 fresh 30×30 chips with 50% degradable microelectrodes and ran 3,000 tasks on each, between six fixed sites. Both routers got the same chips and the same sequence of tasks.

Table 7.2: Chip lifetime over 3,000 tasks (mean $\pm$ std over 10 chips).

Router	Tasks before first failure	Overall success rate
A*	241 $\pm$ 43	0.47 $\pm$ 0.05
Health-aware A*	397 $\pm$ 110	0.72 $\pm$ 0.04

**Figure 7.2:** Chip-lifetime study. A* wears out narrow corridors between the sites; health-aware A* spreads the wear and keeps succeeding for longer, but still declines.

Observations. With health-aware A^* , the chip keeps working without any failure for about 65% longer, and it completes 72% of all tasks compared to 47% for A^* . Figure 7.2(b) shows why: A^* keeps using the same straight corridors between the sites until they die, while health-aware A^* spreads the actuations over a bigger area. But the health-aware curve in Figure 7.2(a) still keeps going down. The router reacts to the *current* health, but it cannot plan for the wear that its own paths are going to cause. A learned policy that optimises over the long term could, at least in principle, do this, and this is the main reason we want a DRL router. We also noticed that “tasks before first failure” varies a lot (± 110), because a single unlucky failure ends the count. The overall success rate is much more stable, so we will mainly use that in later experiments.¹

7.4 Experiment 4 – First PPO Agents

We trained two PPO agents, one with the health channel and one without, for 300k steps each on 12×12 chips with 50% degradable microelectrodes. Then we tested them along with both baselines on 500 chips they had not seen before (Table 7.3).

Table 7.3: Test results on 500 unseen 12×12 chips (50% degradable, $1.5 \times$ deadline).

Method	Success rate	Routing time of successful tasks
A^*	0.644	9.0
Health-aware A^*	0.788	8.8
PPO without health map	0.690	9.1
PPO with health map	0.650	9.2

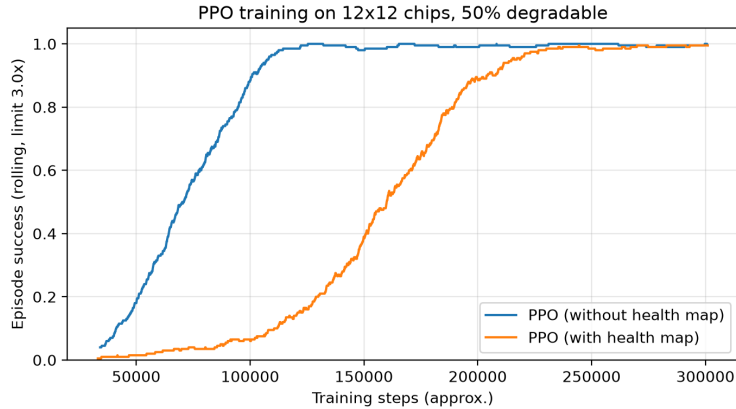


Figure 7.3: Training success of the two PPO agents (rolling average, $3 \times$ training limit).

Observations. Both agents did learn to route. Their training success went from almost zero to about 99% (Figure 7.3), and on unseen chips both are as good as or slightly better than plain A^* . However, neither of them beats health-aware A^* yet, and having the health map has not helped so far. We think there are four main reasons for this:

¹When we ran the same script on Colab, we got 344 ± 46 tasks for health-aware A^* (with overall success again 0.72). Small floating-point differences between library versions change which of two equal-cost paths gets picked on two of the chips. We will report results over multiple seeds in the end-term evaluation.

1. **More input means slower learning.** The agent with the health channel took about twice as many steps to reach 90% training success (around 200k compared to 100k), so by 300k steps it had much less time to improve its policy.
2. **Training limit was too relaxed.** Both agents reach about 99% success under the $3\times$ training limit, so the training signal stops telling apart fast and slow routes. But during testing, the deadline is a much tighter $1.5\times$.
3. **Health only affects the reward indirectly.** The agent only feels the effect of health through failed moves (-0.1), which is a weak and noisy signal. Our simulator already supports adding a small penalty for stepping on weak electrodes, which we plan to try.
4. **Randomness in the results.** With 500 test tasks, the standard error of a success rate around 0.65 is about 2 percentage points, and we only trained with one seed. So the differences between the two PPO agents and A^* are not conclusive yet.

We plan to fix these in the end-term work by training for 1–2 M steps, using a tighter training deadline, adding a health term to the reward and running 3–5 seeds. We have also finished a first curriculum-learning run, but we will report its numbers only after making sure the comparison is fair (same total training steps and same test chips).

8 Conclusion and Future Work

8.1 Conclusion

In this report we presented the first phase of our AI-based, reliability-aware droplet router for MEDA biochips. We built our own MEDA simulator where microelectrodes wear out with use, implemented A^* and a health-aware A^* router whose cost is the expected number of actuation cycles, and trained our first PPO agents. From the experiments we found that: (i) reliability really is a problem, since plain A^* succeeds in only 23% of tasks when 70% of the microelectrodes can degrade; (ii) using health information roughly doubles the success rate on badly degraded chips and makes the chip last about 65% longer; (iii) even health-aware A^* keeps getting worse over time because it cannot predict the wear it causes; and (iv) the PPO agents can already route on unseen chips, but they need more training and a better reward to beat the health-aware A^* baseline. We also think the health-aware A^* router is a useful result by itself, because it gives a fairer baseline to compare DRL against than the health-unaware baselines used in earlier papers.

8.2 Future Work

We have divided the project into five stages: (1) simulator, (2) single-droplet PPO router, (3) curriculum learning, (4) multiple droplets and reshaping, and (5) routing driven by sample preparation. These are spread across four evaluations (Table 8.1). Our immediate next step is to get the DRL router to beat health-aware A^* by training longer, adding a health penalty to the reward and using multiple seeds. The main question we want to answer by the end of Semester 8 is: *which sample-preparation strategy, together with a learned router, keeps a MEDA chip working the longest?* By the end of the project we expect to contribute a stronger health-aware baseline, the first use of curriculum learning for MEDA routing, DRL routing with droplet reshaping, and a study of how sample-preparation workloads like WSPM [26] and REMIA [28] affect chip lifetime.

Table 8.1: Two-semester roadmap.

Milestone	Deliverables
Sem 7 mid-term	Simulator; A* and health-aware A* baselines; first PPO agents; preliminary curriculum run; report and slides.
Sem 7 end-term	PPO trained for 1–2 M steps with a health penalty in the reward, aiming to beat health-aware A*; 3–5 random seeds; full curriculum study on unseen chips; variable droplet sizes; additional baselines (re-planning A*, Dijkstra, a greedy heuristic); fault injection; resource-use and time-per-decision metrics; reservoir/mixer layout for the lifetime test.
Sem 8 mid-term	Multiple droplets with multi-agent PPO and fluidic constraints; reshape actions; conversion of sample-preparation output (WSPM, REMIA, kMTC) into routing tasks; small ILP router to demonstrate its scaling limit.
Sem 8 end-term	Contamination-aware routing; routing complete sample-preparation workloads and measuring assay completion time; main study: which sample-preparation strategy keeps the chip alive longest; final comparison.

References

- [1] M. Elfar, Y.-C. Chang, H. H.-Y. Ku, T.-C. Liang, K. Chakrabarty, and M. Pajic, “Deep reinforcement learning-based approach for efficient and reliable droplet routing on MEDA biochips,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 4, pp. 1212–1222, 2023.
- [2] T.-C. Liang, J. Zhou, Y.-S. Chan, T.-Y. Ho, K. Chakrabarty, and C.-Y. Lee, “Parallel droplet control in MEDA biochips using multi-agent reinforcement learning,” in *Proc. 38th Int. Conf. Machine Learning (ICML)*, PMLR vol. 139, pp. 6588–6599, 2021.
- [3] C. Das, A. Mondal, A. Bose, and B. Das, “A comprehensive literature review on microfluidic biochip technologies: techniques, challenges and future trends,” *Microfluidics and Nanofluidics*, vol. 30, art. 8, 2026.
- [4] X. Liu, Y. Name, and Z. Name, “Exact title of the paper,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. XX, no. X, pp. XXX–XXX, 2026.
- [5] A. Gupta, J.-D. Huang, S. Yamashita, and S. Roy, “Design automation for dilution of a fluid using programmable microfluidic device-based biochips,” *ACM Trans. Design Automation of Electronic Systems*, vol. 24, no. 2, art. 21, pp. 1–24, 2019.
- [6] M. G. Pollack, R. B. Fair, and A. D. Shenderov, “Electrowetting-based actuation of liquid droplets for microfluidic applications,” *Applied Physics Letters*, vol. 77, no. 11, pp. 1725–1726, 2000.
- [7] R. B. Fair, “Digital microfluidics: is a true lab-on-a-chip possible?” *Microfluidics and Nanofluidics*, vol. 3, pp. 245–281, 2007.
- [8] G. Wang, D. Teng, and S.-K. Fan, “Digital microfluidic operations on micro-electrode dot array architecture,” *IET Nanobiotechnology*, vol. 5, no. 4, pp. 152–160, 2011.
- [9] K. Y.-T. Lai, Y.-T. Yang, and C.-Y. Lee, “An intelligent digital microfluidic processor for biomedical detection,” *J. Signal Processing Systems*, vol. 78, pp. 85–93, 2015.
- [10] Z. Li, K. Y.-T. Lai, P.-H. Yu, T.-Y. Ho, K. Chakrabarty, and C.-Y. Lee, “High-level synthesis for micro-electrode-dot-array digital microfluidic biochips,” in *Proc. 53rd Design Automation Conf. (DAC)*, 2016, pp. 1–6.
- [11] Z. Zhong, Z. Li, and K. Chakrabarty, “Adaptive error recovery in MEDA biochips based on droplet-aliquot operations and predictive analysis,” in *Proc. IEEE/ACM ICCAD*, 2017, pp. 615–622.
- [12] H. J. J. Verheijen and M. W. J. Prins, “Reversible electrowetting and trapping of charge: model and experiments,” *Langmuir*, vol. 15, no. 20, pp. 6616–6620, 1999.
- [13] F. Su, W. Hwang, and K. Chakrabarty, “Droplet routing in the synthesis of digital microfluidic biochips,” in *Proc. Design, Automation & Test in Europe (DATE)*, 2006.
- [14] P.-H. Yuh, C.-L. Yang, and Y.-W. Chang, “BioRoute: A network-flow-based routing algorithm for the synthesis of digital microfluidic biochips,” *IEEE Trans. CAD*, vol. 27, no. 11, pp. 1928–1941, 2008.

- [15] T.-W. Huang, C.-H. Lin, and T.-Y. Ho, “A contamination aware droplet routing algorithm for digital microfluidic biochips,” in *Proc. IEEE/ACM ICCAD*, 2009, pp. 151–156.
- [16] Z. Chen, D. H.-Y. Teng, G. C.-J. Wang, and S.-K. Fan, “Droplet routing in high-level synthesis of configurable digital microfluidic biochips based on microelectrode dot array architecture,” *BioChip Journal*, vol. 5, no. 4, pp. 343–352, 2011.
- [17] O. Keszocze, Z. Li, A. Grimmer, R. Wille, K. Chakrabarty, and R. Drechsler, “Exact routing for micro-electrode-dot-array digital microfluidic biochips,” in *Proc. ASP-DAC*, 2017, pp. 708–713.
- [18] P. Howladar, P. Roy, and H. Rahaman, “A high-performance homogeneous droplet routing technique for MEDA-based biochips,” *ACM J. Emerging Technologies in Computing Systems*, vol. 15, no. 4, pp. 1–37, 2019.
- [19] P. Howladar, P. Roy, and H. Rahaman, “Droplet transportation in MEDA-based biochips: an enhanced technique for intelligent cross-contamination avoidance,” *IEEE Trans. VLSI Systems*, vol. 29, no. 7, pp. 1451–1464, 2021.
- [20] C. Das and S. Chakraborty, “An efficient capacity-based routing technique on MEDA-based biochips,” *IETE Journal of Research*, vol. 70, no. 3, pp. 2501–2514, 2024.
- [21] M. Elfar, T.-C. Liang, K. Chakrabarty, and M. Pajic, “Formal synthesis of adaptive droplet routing for MEDA biochips,” *IEEE Trans. CAD*, vol. 41, no. 8, pp. 2504–2517, 2022.
- [22] T.-C. Liang, Z. Zhong, Y. Bigdeli, T.-Y. Ho, K. Chakrabarty, and R. Fair, “Adaptive droplet routing in digital microfluidic biochips using deep reinforcement learning,” in *Proc. 37th ICML*, PMLR vol. 119, pp. 6050–6060, 2020.
- [23] M. Elfar, T.-C. Liang, K. Chakrabarty, and M. Pajic, “Adaptive droplet routing for MEDA biochips via deep reinforcement learning,” in *Proc. DATE*, 2022, pp. 640–645.
- [24] T. Kawakami, C. Shiro, H. Nishikawa, X. Kong, H. Tomiyama, and S. Yamashita, “A deep reinforcement learning approach to droplet routing for erroneous digital microfluidic biochips,” *Sensors*, vol. 23, no. 21, art. 8924, 2023.
- [25] A. Dutta, R. Majumder, and R. K. Pal, “Reinforcement learning based droplet routing technique in hexagonal digital microfluidic biochips using dueling network,” in *Proc. 37th Int. Conf. VLSI Design (VLSID)*, 2024, pp. 67–72.
- [26] Z. Li, K. Y.-T. Lai, K. Chakrabarty, T.-Y. Ho, and C.-Y. Lee, “Droplet size-aware and error-correcting sample preparation using micro-electrode-dot-array digital microfluidic biochips,” *IEEE Trans. Biomedical Circuits and Systems*, vol. 11, no. 6, pp. 1380–1391, 2017.
- [27] T.-C. Liang, Y.-S. Chan, T.-Y. Ho, K. Chakrabarty, and C.-Y. Lee, “Sample preparation for multiple-reactant bioassays on micro-electrode-dot-array biochips,” in *Proc. ASP-DAC*, 2019, pp. 468–473.
- [28] J.-D. Huang, C.-H. Liu, and T.-W. Chiang, “Reactant minimization during sample preparation on digital microfluidic biochips using skewed mixing trees,” in *Proc. IEEE/ACM ICCAD*, 2012, pp. 377–383.
- [29] J.-D. Huang, C.-H. Liu, and H.-S. Lin, “Reactant and waste minimization in multitarget sample preparation on digital microfluidic biochips,” *IEEE Trans. CAD*, vol. 32, pp. 1484–1494, 2013.
- [30] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017.
- [32] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proc. 26th ICML*, 2009, pp. 41–48.
- [33] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *J. Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [34] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv:2407.17032*, 2024.
- [35] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.